

Project Plan: Safe Reinforcement Learning for Graph-Based Agentic Systems

Supervisors: Adel Bibi, Philip Torr (Oxford Part C) **Author:** [Your Name] **Target:** High-quality publication-track research project

0. Important Constraint: No Mid-Project Supervisor Access

Supervisor check-ins are **not available until the project is fully complete**. This changes the execution strategy significantly — self-rigor replaces external feedback loops:

- **Decision Log:** Maintain a running doc logging every major decision (task definition, reward design, method choice) with reasoning. This lets you justify choices at the final review and catch your own drift.
 - **Self-Review Milestones:** At the end of each phase below, do a hard "go/no-go" check — validate results look sane before moving forward. A silent bug compounding for weeks is the single biggest risk of working without checkpoints.
 - **External Feedback Substitutes:** Since the supervisor isn't available mid-project, use other channels for technical sanity checks — emailing authors of closely related papers with specific questions, ML-focused communities (r/MachineLearning, relevant Discords), or informal chats with other RL/ML students if accessible. Don't let this become an echo chamber where only you are checking your own work.
 - Always compare against the unconstrained baseline (Section 5) at every stage — if a "safer" method isn't beating the baseline, something is wrong and needs fixing before moving on, not after.
-

1. Problem Statement

Agentic systems (e.g., an LLM-based calendar assistant) operate over a **state graph**:

- **Nodes** = states of the world (e.g., a scheduled meeting, an empty slot)
- **Edges** = actions that transition between states (e.g., add meeting, reschedule, cancel)

- Each node carries a **safety label**, assigned by an auxiliary LLM-based safety function, indicating whether reaching that state is safe.

Goal: Formulate this as a reinforcement learning (RL) problem where the agent learns a policy that:

1. Maximizes task performance (successfully completes the user's goal)
2. Minimizes/avoids unsafe state transitions
3. Generalizes across different graph structures and safety-labeling functions

This sits at the intersection of **safe RL**, **agentic AI safety**, and **LLM-based reward/constraint modeling** — three of the most active research areas in AI right now.

2. Formalize the Problem (Week 1–2)

Before writing any code, nail the math. This is what separates a "cool demo" from a publishable contribution.

- Define the environment as a **Constrained MDP (CMDP)**:
 - State space $\mathcal{S}$ (graph nodes)
 - Action space $\mathcal{A}$ (graph edges / transitions)
 - Reward function $R(s, a, s')$ — task performance signal
 - Cost/safety function $C(s, a, s')$ — derived from the LLM safety label
 - Constraint: maximize expected reward subject to expected cumulative cost $\leq$ threshold
- Decide how the LLM safety labeler fits in:
 - **Design decision (make this explicit, don't leave ambiguous):** Labels are **precomputed offline** — the entire graph is labeled once by the LLM safety function before RL training begins. This is preferred over online/per-step querying because it (a) matches the abstract's framing of labels as static node properties, (b) makes training reproducible and removes LLM non-determinism from the training loop itself, and (c) makes cost $C(s, a, s')$ a fixed, known function during training — turning this into a standard CMDP rather than an online-safety-estimation problem. Document this choice and its justification here; it is foundational and everything downstream depends on it.
 - **Constraint type (also decide explicitly):** Choose between a **hard constraint** (visiting an unsafe state terminates the episode / incurs a large penalty) versus a **soft/cumulative constraint** (unsafe visits accumulate cost, constrained to stay under a threshold — standard CMDP style). Recommend starting with the soft/cumulative formulation since it's what Lagrangian methods and CPO are designed for, and is more consistent with "guide risk-aware decision-making" language in the abstract (a spectrum of risk, not a binary pass/fail). This choice must be fixed before implementing any safe-RL method, since Lagrangian and shielding approaches are implemented differently depending on it.
- Is the label binary (safe/unsafe) or a continuous risk score?
- Write a one-page formal spec (LaTeX). This becomes the skeleton of your eventual paper's Section 3 ("Problem Formulation").

Deliverable: A written formalization doc you can hand to your supervisor in your first meeting — signals seriousness immediately.

3. Literature Review (Week 1–3, parallel)

Read and take structured notes (not just skim) on:

Area	What to look for
Safe RL	CMDPs, Lagrangian methods, shielding, constrained policy optimization (CPO), safe exploration
Agentic AI safety	LLM agents in tool-use/graph environments, failure modes, benchmarks (e.g. BAD-ACTS, Risky-Bench style work)
LLM-as-judge / reward modeling	Using LLMs to assign safety/reward labels, reliability and noise of these labels
Graph-based RL	Navigating graph-structured state spaces, GNN-based policies if relevant

Deliverable: A living annotated bibliography (Notion/Obsidian/plain markdown) — 15–25 papers, each with a 2-3 line "what's the idea, what's missing, how does it relate to my project" note. This becomes your paper's Related Work section later.

3b. Define the Concrete Task (Week 2)

"Task performance" needs a precise definition before reward design is possible.

- Write 3–5 concrete user scenarios, e.g.:
 - "Schedule a 1-hour meeting with Person X sometime this week, avoiding conflicts"
 - "Reschedule an existing meeting to accommodate an urgent request"
 - "Cancel a low-priority meeting if a higher-priority one conflicts"
- These scenarios drive how synthetic graphs are generated — purely random graphs won't produce realistic or interpretable results.
- From each scenario, define a clear success condition (what does "task completed" mean numerically, for the reward function).

Deliverable: A short scenario spec doc — this becomes part of your Method section later.

3c. Validate the Safety Labeler — AND Treat Labeler Choice as a Core Experimental Variable

The abstract explicitly frames the safety label as coming from "**potentially another** LLM-based safety function" — this phrasing means the labeler itself is a variable to study, not a fixed, trusted component. Two distinct pieces of work follow from this:

(i) One-time validation (sanity check, do this early):

- Hand-annotate a subset of states (50–100) yourself, and compare against LLM-generated labels — compute agreement (e.g., Cohen's kappa).
- Document disagreement cases — these often reveal ambiguity in what "safe" even means in this context, useful for the paper's discussion.

(ii) Labeler sensitivity as a core ablation (a primary experiment, not a footnote):

- Generate labels using multiple different LLMs (e.g., Claude, GPT, a smaller open-source model) and measure how much the learned safe-RL policy's performance and safety profile change depending on which labeler produced the training labels.
- Separately, inject controlled synthetic noise into labels (e.g., flip 5%, 10%, 20% of labels at random) and measure how gracefully each safe-RL method degrades. **This robustness-to-label-noise result is likely your most novel and citable contribution** — it directly operationalizes the abstract's "potentially" framing into a real experimental finding.

Deliverable: A validation report (i) plus a full labeler-sensitivity results section (ii) with plots showing performance/safety vs. labeler choice and vs. noise level.

4. Build the Environment (Week 2–4)

This is your core engineering deliverable and the foundation everything else sits on.

- Use **Gymnasium** to build a custom environment: `CalendarGraphEnv`
 - Nodes = calendar states (slot empty, meeting X scheduled, etc.)
 - Actions = add/reschedule/cancel
 - Start simple: small synthetic graphs (10–50 nodes) before scaling up
- Build a **safety labeling module**:
 - Wrap an LLM API call (Claude/GPT) that takes a state description and returns safe/unsafe (+ optionally a rationale)
 - Cache labels aggressively — LLM calls are slow/expensive during RL training with millions of steps
 - Consider a **distilled/cheap surrogate model** (e.g., small classifier trained on LLM-generated labels) to make training tractable — this itself could be a nice contribution
- Log everything from day one (Weights & Biases or plain CSV) — reproducibility matters for a publication.

Deliverable: A working environment where a random policy can act, with safety labels visible per state, and a clean `README` describing it. This is what you show your supervisor as your "proof of concept."

4b. Verify RL Implementation on an Established Benchmark (before trusting results on your own environment)

Before running any safe-RL method on the novel `CalendarGraphEnv`, validate your own implementation against a known, published safe-RL benchmark (e.g., **Safety-Gymnasium**).

- Reproduce a known result (approximately) on the benchmark using your PPO / Lagrangian-PPO code.
 - Only once this matches published numbers reasonably well should you trust results on your own environment.
 - **Why this matters:** on a brand-new environment, if results look strange there's no way to tell whether it's a genuine finding or a bug in your RL code. This step separates the two failure modes early, before you've built an entire narrative on top of a bug.
-

5. Baseline Agents (Week 4–6)

- Implement an **unconstrained baseline**: standard PPO (via `Stable-Baselines3`) optimizing task reward only, ignoring safety. This is your "what happens without safety constraints" reference point — you need this to demonstrate the problem is real.
- Implement a **naive safety filter**: reject/mask unsafe actions at the action-selection step (a simple non-RL baseline).
- Measure: task success rate, unsafe-state visitation rate, episode length.

Deliverable: First results table/plot — even mediocre numbers here are useful because they define the problem empirically.

6. Safe RL Methods (Week 6–10)

This is the technical core of the project. Pick 2–3 methods to implement and compare, not just one:

1. **Lagrangian-constrained PPO** — augment the reward with a learned penalty for cost violations
2. **Safety shielding / action masking with learned safety critic** — learn to predict unsafe transitions and mask them proactively rather than reactively
3. (Stretch) **Risk-aware / distributional RL** — model the full distribution of outcomes, not just the mean, for more conservative decision-making

Deliverable: Working implementations, ablations (e.g., how does performance change as you vary the safety threshold, or as the LLM labeler gets noisier — see Section 6b for the fairness protocol governing all of these).

6b. Fairness-Controlled Ablation Protocol (governs every comparison in this project)

This is the backbone of experimental credibility — every comparison between methods must follow this protocol, or results are not trustworthy:

- **Equal tuning budget:** every method (baseline PPO, Lagrangian-PPO, shielding, distributional RL) gets the same hyperparameter search budget (e.g., same number of random search trials). An undertuned baseline that "loses" to your proposed method proves nothing — this is one of the most common ways safe-RL papers get criticized.
 - **One variable per ablation:** each ablation study changes exactly one factor (e.g., safety threshold, graph size, labeler identity, label noise level) while holding everything else fixed — algorithm, hyperparameters, number of training steps, evaluation graphs.
 - **Multiple seeds, always:** run every configuration with at least 5 random seeds and report mean $\pm$ confidence interval, never a single-run number.
 - **Statistical significance:** when claiming one method beats another, back it with a statistical test (e.g., bootstrap confidence intervals or a paired t-test across seeds), not just a higher mean.
 - **Fixed evaluation protocol:** all methods are evaluated on the exact same held-out set of graphs/scenarios (see Section 6c) with the same number of evaluation episodes.
-

6c. Train/Test Split for Generalization

- Generate a pool of synthetic calendar graphs, and hold out a subset (unseen during training) purely for evaluation — never train and test on the same graphs.
 - Vary held-out graphs along axes not seen in training where possible (different sizes, different scenario mixes) to test whether the learned policy generalizes or has just memorized training-graph quirks.
 - Report both in-distribution (held-out but similar) and out-of-distribution (structurally different) results separately — this distinction itself is a meaningful, reportable finding.
-

7. Evaluation & Benchmarking (Week 10–12)

- Build 2–3 graph environments of increasing complexity/realism (not just calendars — maybe a second domain like a simple task-planning graph, to show generality)
- Metrics: task success rate, cumulative safety violations, sample efficiency, robustness to label noise
- **Stress-test the LLM safety labeler itself** — how much does the whole system degrade if the labeler is wrong X% of the time? This is a genuinely interesting and citable finding.

Deliverable: A benchmark suite + results that could stand alone as a contribution even before the RL method itself.

8. Writing & Publication Track (Week 10–14, parallel with above)

- Target venue: workshop papers at NeurIPS/ICML/ICLR (Safe RL, Agentic AI, or Trustworthy ML workshops) are realistic first targets; a full conference paper is the stretch goal.
- Structure early: Introduction → Related Work (from step 3) → Problem Formulation (from step 2) → Method → Experiments → Discussion/Limitations
- Write the **Limitations** section honestly — reviewers respect this, and it's standard in top venues now.

Deliverable: Full draft, self-reviewed against the Section 0 protocol, ready to present as a complete package at the final supervisor review.

8b. Reproducibility & Non-Determinism

LLM APIs are non-deterministic, and RL training involves millions of steps — small inconsistencies compound.

- Use temperature=0 for the safety labeler where possible, but still test for output variance across repeated calls on the same input.
 - Fix random seeds and report results across multiple runs (3–5 seeds minimum) — single-run results are not accepted at top venues.
 - Estimate compute budget (GPU-hours) early and confirm access to Oxford's compute cluster before it becomes a bottleneck.
-

8c. Ethics & Broader Impact

This project concerns safety-critical AI decision-making — top venues (NeurIPS/ICML/ICLR) now expect a Broader Impact or Ethics statement as standard.

- Discuss real-world deployment risks: could this approach create a false sense of safety? Could the RL agent overfit to quirks/biases of the specific LLM labeler used?
 - Start drafting this section early rather than as an afterthought — it should reflect genuine engagement with limitations, not a token paragraph.
-

9. Suggested Timeline Overview

Weeks	Focus
1–2	Formalization + start literature review
2–4	Build environment + safety labeler
4–6	Baselines
6–10	Safe RL methods + ablations

Weeks	Focus
10–12	Benchmarking + robustness analysis
10–14	Writing (parallel)
Final weeks	Polish, supervisor review rounds, submission prep

Adjust against your actual Part C deadline calendar — work backward from the hard submission date.

10. Tech Stack

- **Python, PyTorch**
- **Gymnasium** for the environment
- **Stable-Baselines3** (or a custom PPO loop if you need more control for the Lagrangian variant)
- **Claude/GPT API** for the safety labeler (cache responses!)
- **Weights & Biases** for experiment tracking
- **Git** from day one — commit often, tag milestone versions
- **LaTeX** (Overleaf) for the writeup, started early, not at the end

11. Risk Mitigation

- **LLM API costs/rate limits**: cache aggressively, consider a distilled local classifier early
- **RL training instability**: start with small, simple graphs before scaling; sanity-check with the unconstrained baseline first
- **Scope creep**: the CMDP + 2 safe-RL methods + benchmark is already a full project — resist adding a third domain or a fourth method unless the core is solid
- **No mid-project supervisor access**: since checkpoints with Bibi/Torr aren't available until the end, rely on the Section 0 protocol (decision log, self-review milestones, external community feedback) to catch problems early instead of a supervisor catching them for you

Immediate Next Actions

1. Write the one-page formalization doc (Section 2) — including the precomputed-labeling and soft-constraint decisions — and log it as your first Decision Log entry
2. Set up the GitHub repo + environment skeleton this week
3. Start the annotated bibliography in parallel

